

AI/ML Engineer - RAG & GenAI Interview Prep

50 interview questions with concise, senior-level answers for a 5-year experience profile

How to use this guide

First try answering each question aloud in 60-90 seconds. Then compare your response with the model answer. For senior-level interviews, explain trade-offs, failure modes, metrics, security, and production decisions rather than only defining terms.

Recommended interview pattern: Definition -> architecture/approach -> trade-off -> production example -> metric or validation method.

1. RAG Fundamentals

Q1. What is Retrieval-Augmented Generation (RAG), and why is it useful?

Interview-ready answer: RAG combines information retrieval with an LLM. At query time, the system retrieves relevant external documents or chunks and places them into the model context so the answer is grounded in current or private knowledge. It is useful because knowledge can be updated without retraining the model and responses can include source citations.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q2. Explain an end-to-end RAG architecture.

Interview-ready answer: A typical flow is: document ingestion -> parsing/cleaning -> chunking -> embeddings -> vector or hybrid index -> user query -> query processing -> retrieval -> reranking/filtering -> prompt construction -> LLM generation -> citations/response -> logging and evaluation. Production systems also add access control, caching, observability, and fallback behavior.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q3. How is RAG different from fine-tuning?

Interview-ready answer: Fine-tuning changes model parameters so the model learns behavior, style, or task patterns. RAG leaves model weights unchanged and supplies external knowledge at inference time. For frequently changing enterprise facts, RAG is usually preferable; fine-tuning is more useful for behavior, format, domain style, or task specialization.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q4. What is an embedding in a RAG system?

Interview-ready answer: An embedding is a dense numeric vector representing the semantic meaning of text. Similar meanings tend to be close in vector space, which enables semantic retrieval even when the query and document use different words. Embedding quality, dimensionality, language coverage, and domain fit directly affect retrieval quality.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q5. What is semantic search?

Interview-ready answer: Semantic search retrieves content based on meaning rather than exact keyword overlap. The query and candidate chunks are embedded and compared using a similarity function. It is strong for paraphrases and natural-language questions, but keyword or exact-match retrieval can still be better for identifiers, error codes, product IDs, and rare terms.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q6. Why is chunking important in RAG?

Interview-ready answer: Chunking determines the units that are embedded and retrieved. Chunks that are too large may contain irrelevant material and waste context; chunks that are too small may lose the surrounding meaning. Good chunking preserves semantic boundaries while keeping retrieved context focused and sufficient.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q7. How do you choose chunk size and overlap?

Interview-ready answer: There is no universal size. I start with document structure and an overlap large enough to preserve context across boundaries, then evaluate retrieval and answer quality on a representative test set. I would tune chunk size, overlap, separator strategy, and metadata together rather than choosing values arbitrarily.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q8. What is cosine similarity, and why is it common for embeddings?

Interview-ready answer: Cosine similarity measures the angle between two vectors and is largely insensitive to vector magnitude. For normalized embeddings it works well as a semantic similarity measure. In practice, the correct metric depends on the embedding model and vector database configuration, so I verify whether the model expects cosine, dot product, or another distance.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q9. Compare cosine similarity, dot product, and Euclidean distance.

Interview-ready answer: Cosine measures orientation; dot product considers both orientation and magnitude; Euclidean distance measures geometric distance. With normalized vectors, cosine similarity and dot product become closely related. The correct choice should match the embedding model and indexing setup rather than being selected by habit.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q10. How would you choose a vector database for a production RAG system?

Interview-ready answer: I would compare retrieval quality, filtering capabilities, hybrid search support, scale, indexing and query latency, operational complexity, availability, security, and cost. Examples include managed vector stores and systems built on Elasticsearch/OpenSearch, PostgreSQL with pgvector, Milvus, Qdrant, Pinecone, or cloud-native alternatives. The best choice depends on workload and existing platform standards.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

2. Retrieval and Query Engineering

Q11. What is metadata filtering, and why is it important?

Interview-ready answer: Metadata filtering restricts retrieval using fields such as tenant, document type, product, region, date, or access level. It improves both relevance and security. For enterprise RAG, authorization filters must be enforced before or during retrieval so unauthorized chunks never reach the LLM context.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q12. What is hybrid search?

Interview-ready answer: Hybrid search combines lexical retrieval, such as BM25, with semantic vector retrieval. Lexical search is strong for exact terms, identifiers, and rare words; vector search is strong for semantic similarity. Combining them often improves recall and robustness, especially in technical or enterprise corpora.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q13. Why can pure vector search fail?

Interview-ready answer: Vector search can miss exact identifiers, numbers, product codes, acronyms, and terms with weak representation in the embedding model. It can also retrieve semantically similar but factually wrong content. Hybrid retrieval, metadata filters, better chunking, reranking, and evaluation can address these weaknesses.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q14. What is reranking and where does it fit?

Interview-ready answer: A first-stage retriever quickly returns a candidate set, such as the top 20 or 50 chunks. A reranker then scores those candidates with a stronger cross-encoder or reranking model using both query and candidate text. The system passes only the best few chunks to the LLM, improving precision at the cost of extra latency and compute.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q15. How do you choose top-k for retrieval?

Interview-ready answer: I treat k as a tunable system parameter. A larger k usually increases recall but also adds noise, latency, and context cost. I would evaluate recall@k, context relevance, answer quality, latency, and token cost, then select the smallest k that meets the application's quality target.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q16. What is query rewriting and when is it useful?

Interview-ready answer: Query rewriting transforms a user question into a form that retrieves better evidence. Examples include adding missing context from conversation history, expanding acronyms, generating search-friendly phrasing, or decomposing a complex question. It helps when the user's original wording is ambiguous or not retrieval-friendly.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q17. How would you handle a multi-turn conversational RAG system?

Interview-ready answer: I separate conversation understanding from retrieval. First, I rewrite the current turn into a self-contained search query using relevant history. Then I retrieve documents and generate an answer grounded in those documents. I also manage conversation memory carefully so irrelevant history does not consume the context window.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q18. What is parent-child chunking?

Interview-ready answer: Parent-child chunking stores smaller child chunks for precise retrieval while retaining a larger parent section for context. Retrieval finds the child, then the system can return the parent or adjacent context to the LLM. This balances retrieval precision with contextual completeness.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q19. How should RAG handle tables and structured documents?

Interview-ready answer: Naively splitting tables into text can destroy row-column relationships. I would use structure-aware parsing, preserve headers and row context, and consider converting tables into normalized representations when appropriate. For questions requiring calculations or exact filtering, I may route the request to a structured query tool instead of relying only on text retrieval.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q20. Describe a robust PDF ingestion pipeline for RAG.

Interview-ready answer: I would detect file type and extract text with layout awareness, preserve headings and metadata, identify tables and images, and use OCR for scanned pages when needed. Then I normalize text, remove duplicates, chunk by structure, attach metadata such as page number and section, create embeddings, and index the chunks. I would validate extraction quality before production indexing.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

3. Data, Indexing, and Retrieval Quality

Q21. How would you handle scanned PDFs or image-heavy documents?

Interview-ready answer: I would detect pages with little or no extractable text and route them through OCR. For layout-heavy material, I would preserve coordinates or page references where possible. OCR output should be quality-checked because recognition errors can directly reduce retrieval accuracy.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q22. How do deduplication and document versioning affect RAG?

Interview-ready answer: Duplicate content can distort retrieval and waste context, while stale versions can cause conflicting answers. I would assign stable document IDs and version metadata, deduplicate by content hash or canonicalized text, and mark active versions explicitly. Re-indexing should be incremental rather than rebuilding everything whenever possible.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q23. What metadata would you store with each chunk?

Interview-ready answer: At minimum I would store document ID, version, source URI, title, section, page or location, timestamps, tenant or security scope, document type, language, and chunk order. Good metadata enables filtering, citations, debugging, lifecycle management, and access control.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q24. How would you implement tenant isolation in a multi-tenant RAG system?

Interview-ready answer: Every document and chunk should carry tenant identity and authorization metadata. Retrieval must enforce tenant filters server-side, not depend on the LLM prompt. I would also isolate caches, logs, indexes or namespaces as appropriate and test cross-tenant leakage explicitly.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q25. What is the difference between retrieval precision and retrieval recall?

Interview-ready answer: Retrieval recall asks whether the system found the relevant evidence that exists in the corpus; retrieval precision asks how much of what it retrieved is actually relevant. A high-recall first stage followed by reranking is a common production pattern because it avoids missing the answer while controlling final-context noise.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q26. How would you evaluate a retriever independently from the LLM?

Interview-ready answer: Create a labeled query set with relevant documents or chunks and measure metrics such as Recall@k, Precision@k, MRR, nDCG, or hit rate. This isolates retrieval problems from generation problems. I would evaluate across query types, document categories, and difficult edge cases rather than relying on one aggregate metric.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q27. What are the main metrics for evaluating a RAG application?

Interview-ready answer: I separate retrieval metrics from generation metrics. Retrieval can use Recall@k, Precision@k, MRR, and nDCG; generation can use groundedness or faithfulness, answer relevance, completeness, and citation correctness. I also monitor latency, token usage, failure rate, and user feedback because business quality is multidimensional.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q28. How do you build a good RAG evaluation dataset?

Interview-ready answer: I would create representative questions from real user tasks, include easy and hard cases, and label the relevant evidence and expected answer characteristics. The set should include ambiguous queries, exact-match queries, multi-hop questions, no-answer cases, and adversarial cases. It should be versioned so system changes can be compared consistently.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q29. What is the difference between offline and online evaluation for RAG?

Interview-ready answer: Offline evaluation uses a fixed benchmark or golden dataset before deployment and is useful for regression testing. Online evaluation uses production signals such as user feedback, click behavior, escalation rate, answer acceptance, and sampled human review. Mature systems use both because offline tests provide repeatability while online metrics reveal real-world drift.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q30. What are the limitations of LLM-as-a-judge evaluation?

Interview-ready answer: An LLM judge can scale evaluation, but it may have bias, inconsistent scoring, sensitivity to wording, or agreement with fluent but incorrect answers. I would calibrate it against human judgments, use explicit rubrics and pairwise comparisons where useful, and keep deterministic retrieval/citation checks for important claims.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

4. Generation, Prompting, and Context Management

Q31. How do you reduce hallucinations in a RAG system?

Interview-ready answer: I improve retrieval quality, restrict the generation prompt to provided evidence, require the model to say when evidence is insufficient, and validate important claims or citations. I also use metadata filters, reranking, answerability checks, and evaluation datasets that include no-answer cases. Hallucination control is an end-to-end system problem, not just a prompt problem.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q32. How would you design a RAG prompt?

Interview-ready answer: The prompt should clearly separate system instructions, user question, and retrieved evidence. I specify that the model should answer only from the supplied sources when grounding is required, preserve uncertainty when evidence is missing, and return citations in a defined format. I keep instructions concise and consistent to reduce ambiguity.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q33. How do you manage context-window limits?

Interview-ready answer: I control the amount of retrieved text using top-k tuning, reranking, compression, and selective context expansion. I remove redundant chunks and avoid inserting entire documents when smaller evidence is enough. I also track input-token cost because a larger context can increase both cost and latency without improving answer quality.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q34. What is the 'lost in the middle' problem?

Interview-ready answer: LLMs can use information at the beginning and end of long contexts more reliably than information buried in the middle. To reduce this, I keep only high-value chunks, order evidence intentionally, and use retrieval and compression to keep contexts concise. Evaluation should include long-context cases because a large context window does not guarantee effective use of all content.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q35. What is query expansion?

Interview-ready answer: Query expansion generates additional terms or reformulations to improve recall. For example, a technical acronym can be expanded to its full name, or several semantic variants can be searched. I would use it selectively because excessive expansion can increase latency and introduce irrelevant candidates.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q36. What is multi-query retrieval?

Interview-ready answer: Multi-query retrieval creates several alternative search queries from the user's question, retrieves for each, and combines the results before reranking. It can improve recall for ambiguous or multi-faceted questions. I would deduplicate results and control the number of generated queries to manage cost and latency.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q37. What is HyDE (Hypothetical Document Embeddings)?

Interview-ready answer: HyDE asks the LLM to generate a hypothetical answer or document and then embeds that generated text for retrieval. The idea is that the hypothetical document may contain terms closer to the target corpus than the original short query. It can help difficult semantic retrieval cases, but it adds model latency and may introduce retrieval bias.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q38. What is Maximal Marginal Relevance (MMR)?

Interview-ready answer: MMR balances relevance with diversity. Instead of selecting the top similar chunks only, it penalizes candidates that are too similar to already selected chunks. This can reduce duplicate evidence and give the LLM broader coverage of a user's question.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q39. How would you generate reliable citations from RAG?

Interview-ready answer: Each chunk should carry stable provenance such as document ID, title, URL, section, and page. The generation layer should associate claims with the retrieved evidence, and the application can render citations from the source metadata. I would test citation correctness separately because a citation existing in the answer does not guarantee that it actually supports the claim.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q40. How do you handle questions that are not answerable from the knowledge base?

Interview-ready answer: I add an answerability or confidence check and instruct the model not to invent facts. The response should clearly state that the available sources are insufficient and, where appropriate, ask a clarifying question or route to another system. No-answer evaluation cases are essential to prevent forced hallucinations.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

5. Production RAG, Security, and System Design

Q41. How would you design caching for a RAG system?

Interview-ready answer: I can cache embeddings, retrieval results, reranking results, and sometimes final responses depending on freshness requirements. Cache keys should include relevant model versions, query normalization, tenant/security scope, and document index version where necessary. I also need explicit invalidation rules for updated or deleted documents.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q42. How would you reduce RAG latency in production?

Interview-ready answer: I parallelize independent work, use efficient vector indexes, keep top-k modest, rerank only a small candidate set, cache repeat operations, and stream the final response. I profile each stage separately because a slow parser, network call, embedding model, vector search, reranker, or LLM can dominate total latency.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q43. How would you optimize RAG cost?

Interview-ready answer: I control embedding and generation token usage, avoid unnecessary re-embedding, cache repeated queries, rerank only where it adds measurable quality, and choose smaller models for routing or rewriting tasks. Cost should be measured together with quality, latency, and user value rather than minimized in isolation.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q44. What observability would you add to a production RAG pipeline?

Interview-ready answer: I would trace the request end-to-end and capture retrieval queries, selected document IDs, similarity scores, filters, reranker scores, prompt versions, model latency, token usage, errors, and final citations. Sensitive content must be redacted appropriately. This makes it possible to distinguish retrieval failures, prompt failures, model failures, and infrastructure failures.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q45. How do you protect a RAG system from prompt injection in retrieved documents?

Interview-ready answer: Retrieved text must be treated as untrusted data, not as instructions. System prompts should explicitly separate trusted instructions from retrieved content, and tool calls or privileged actions should require additional authorization and validation. I would also test malicious documents that attempt to override instructions, exfiltrate secrets, or manipulate downstream tools.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q46. How do you prevent PII or confidential data leakage in RAG?

Interview-ready answer: Enforce authorization before retrieval, apply tenant and document-level access filters, and minimize sensitive data in prompts and logs. I would add DLP or PII detection where appropriate, encryption, retention controls, and access auditing. Security testing should include attempts to retrieve unauthorized documents through indirect or conversational queries.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q47. How do you handle stale data in a RAG system?

Interview-ready answer: I attach document timestamps and version information, define freshness requirements, and support incremental updates and deletions. For time-sensitive domains, retrieval can prioritize recent versions or apply date filters. A background indexing pipeline should detect source changes and propagate them reliably to the search index.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q48. What fallback strategy would you use when retrieval or the LLM fails?

Interview-ready answer: The system should fail gracefully. If retrieval is unavailable, I may return a transparent error or a limited response rather than fabricate an answer. If retrieval succeeds but evidence is insufficient, use a no-answer response. For transient infrastructure failures, retries with timeouts and circuit breakers are safer than unbounded retries.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q49. Design an enterprise RAG system that supports millions of documents and many users.

Interview-ready answer: I would build an asynchronous ingestion pipeline, object storage for raw files, a metadata/catalog layer, chunking and embedding workers, a scalable vector/hybrid search tier, and stateless query services. Retrieval would enforce tenant authorization and use first-stage search plus reranking. The serving layer would include caching, observability, rate limits, model routing, citations, and automated evaluation, with index updates handled incrementally and idempotently.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.

Q50. A RAG application is returning incorrect answers. How would you debug it?

Interview-ready answer: I would trace one failing query end-to-end and separate retrieval from generation. First check whether the correct source exists, whether ingestion and chunking preserved it, whether filters removed it, and whether retrieval/reranking selected it. If the right evidence is present, inspect prompt construction, context ordering, model behavior, citations, and answerability rules. Then reproduce the case in the evaluation set and add a regression test so the fix is measurable and permanent.

Senior cue: In a real interview, add one concrete project example and mention the metric or trade-off you used.